DAAP HMW1: LPC-10 Speech Coding

Report for Speak & Spell Simulation

Authors:
Francesco Moretti (270077)
francesco14.moretti@mail.polimi.it

Luca Trapella (279894)
luca.trapella@mail.polimi.it

April 2025

Contents

1. Introduction

This report describes the implementation of the LPC-10 based speech coding assignment in MATLAB. The coding approach simulates the Speak & Spell toy by Texas Instruments. The approach was modular, with separate MATLAB functions for encoding, pitch detection, voiced/unvoiced frame detection, excitation signal generation, and decoding. Key design choices for this implementation are:

- A sampling rate of 8 kHz to match the original hardware limitations.
- Different LPC orders for voiced ($P = 10$) and unvoiced ($P = 4$) frames to reflect the spectral complexity.
- Use of pre-emphasis filtering and windowing to prepare the speech signal.

The `lpc_test.m` script serves as the main driver for the entire LPC encoding and decoding process. It orchestrates the workflow by:

- Selecting the file to process.
- Running the encoder to generate the LPC parameters.
- Calling the decoder to reconstruct the speech signal.
- Measuring the quality of reconstruction via the Signal-to-Noise Ratio (SNR) and the compression capabilities of the LPC-10.

2. LPC Encoder

The start of the `encoder.m` script handles preparing the input audio. It begins by reading the audio file and then continues with pre-processing steps. These steps include changing the audio sample rate to 8000 Hz and applying pre-emphasis filters. Doing this makes the audio suitable for the LPC-10 algorithm. The 8000 Hz rate is essential because the algorithm is designed to work at this rate. Additionally, converting the audio to mono (a single channel) helps avoid any issues that might arise from handling stereo or multi-channel audio. This process ensures that the LPC analysis, which works best with mono signals, functions as intended. Normalization scales the signal within a controlled range, enhancing numerical stability and consistency across processing steps.

Splitting the signal into frames is important because the nature of speech changes over time. When we divide audio into short, overlapping frames, each is treated as stable, making analysis and processing easier. During reconstruction, overlap-and-add combines these frames smoothly, preventing breaks or unwanted noises at the edges where frames meet.

In this analysis, we use a 256 sample Hamming windoew with a hop size of 128 samples. This choice offers a good balance between time and frequency resolution: at a sampling rate of 8000 Hz, the window spans 32 ms. That is short enough to track speech dynamics, yet long enough to capture relevant spectral information for both male and female voices. The hop size, being half the window length, satisfies the Constant Overlap-Add (COLA) condition for perfect reconstruction.

To compute the Discrete Fourier Transform (DFT), we use an FFT size of $N_{\text{fft}} = 1024$, which introduces zero-padding (appending zeros to the 256 sample frame before computing the FFT). While this does not improve the true frequency resolution, that depends solely on the window length, it increases the number of frequency bins, providing finer spectral detail and improving peak localization (more accuracy).

At 8000 Hz, the frequency bin spacing becomes:

$$\Delta f = \frac{F_s}{N_{\text{fft}}} = \frac{8000}{1024} \approx 7.8 \text{ Hz}$$

This denser frequency grid enhances the visual clarity of the spectrum and allows better matching to the spectral envelope estimated by the LPC filter.

The main lobe width B_w introduced by the window function can be estimated as:

$$B_w = \frac{L \cdot F_s}{M}$$

where:

- L is a constant depending on the window type:
  - Rectangular: $L = 2$
  - Hamming/Hanning: $L = 4$
  - Blackman: $L = 6$
- M is the window length (in samples)
- F_s is the sampling rate (in Hz)

To resolve two sinusoids separated by a frequency difference Δf, a sufficient condition is:

$$M \geq \frac{L \cdot F_s}{\Delta f}$$

For a male voice with $f_0 = 100$ Hz, the harmonics are spaced by $\Delta f = 100$ Hz. Using a Hanning window ($L = 4$), the minimum required window length becomes:

$$M \geq \frac{4 \cdot 8000}{100} = 320 \text{ samples}$$

Our current frame size of 256 samples (32 ms) corresponds to approximately:

$$\frac{256}{80} = 3.2 \text{ periods}$$

with a period $P = \frac{F_s}{f_0} = 80$ samples. While slightly below the theoretical minimum for resolving harmonics at 100 Hz, this setup remains practically effective for typical speech analysis. For low-pitched voices, where harmonics are closely spaced, longer windows may be required for higher-resolution spectral modeling.

For each frame, the following steps are performed:

1. **Voiced/Unvoiced detection:** Determine if the frame is voiced or unvoiced (see Section 2.1 for details). This classification is crucial because it influences the Linear Predictive Coding (LPC) order and other parameter computations.

2. **Whitening filter:** Compute the whitening filter coefficients using the `levinson` function. The Levinson-Durbin algorithm is used to solve the Yule-Walker equations for linear prediction, which estimates the coefficients of an autoregressive (AR) model.

   - **Input:**
     - `r_pos`: A vector of autocorrelation coefficients $r(0), r(1), \ldots, r(P)$, where $r(k)$ is the autocorrelation at lag k. These coefficients must be from a positive-definite sequence (hence the name `r_pos`). Putting $r_p(1) = eps$ if $r_p(1) = 0$, ensures numerical stability in the Levinson algorithm (preventing NaNs). $r(0) = 0$ would imply zero energy, making the autocorrelation matrix singular.
     - `p`: The order of the AR model (the number of coefficients to compute). The LPC order P is adapted based on the voiced/unvoiced classification for each frame:
       - **Voiced frames:** $P = 10$. A higher order is used to capture the rich harmonic content and finer spectral details typical of voiced speech.
       - **Unvoiced frames:** $P = 4$. A lower order suffices since unvoiced frames have less spectral structure and require fewer coefficients.
   - **Output:**
     - `a`: The AR coefficients $[1, a_1, a_2, \ldots, a_P]$ of the whitening filter. These coefficients minimize the prediction error in the least squares sense.
     - `error`: The prediction error variance (scalar), which represents the MSE of the forward prediction. This is proportional to the power of the whitened signal.

3. **Prediction error:** Compute the prediction error using the `filter` function. The prediction error represents the residual signal after removing the predictable components, which is essential for compression.

4. **Mean Squared Error (MSE):** Calculate the MSE of the frame, which measures the energy of the prediction error and is used for further analysis and gain computation.

Additionally, the following parameters are computed and passed to the decoder:

- **Voiced Frames:**

- **Pitch:** The pitch period is estimated (see Subsection 2.3 for details).
- **Gain:** To ensure faithful synthesis, the power of the synthesized excitation signal must match the power of the prediction error signal. Therefore, the gain must be computed such that this condition is satisfied.

  First, we compute the number of samples within the analysis window that correspond to an integer multiple of the pitch period:

$$\texttt{lim} = \left\lfloor \frac{\texttt{winLen}}{\texttt{pitch}} \right\rfloor \cdot \texttt{pitch}$$

  This ensures that we consider a whole number of pitch periods for accurate energy estimation.

  Then, the power of the prediction error signal over these `lim` samples is computed as:

$$\texttt{power} = \frac{1}{\texttt{lim}} \sum_{n=1}^{\texttt{lim}} e[n]^2$$

  where $e[n]$ is the prediction error for the current frame.

  The synthesized excitation signal is modeled as a train of impulses with amplitude `gain` and period `pitch`. Its average power is:

$$P_{\text{synth}} = \frac{1}{N} \sum_{n=0}^{N-1} e[n]^2 = \frac{\texttt{gain}^2}{\texttt{pitch}}$$

  Equating the two powers and solving for `gain`, we obtain:

$$\texttt{gain} = \sqrt{\texttt{power} \cdot \texttt{pitch}}$$

- **Unvoiced Frames:**
  - **Gain:** the gain is computed as the square root of the corresponding MSE, providing a measure of the frame's energy.

Finally, all necessary parameters (LPC coefficients, pitch, gain, etc.) are saved for use in the decoding phase.

2.1 Voiced frame detection with ZCR

The `voicedframedetection.m` script determines whether each frame is voiced or unvoiced. The decision is based on two key features:

- **Zero-Crossing Rate:** The ZCR is a measure of the frequency content of a signal and is particularly usefull for detecting voiced or unvoiced frames. It counts how often the signal changes sign (crosses the zero axis) within a frame. The ZCR is computed as:

$$\texttt{zcr} = \frac{1}{2} \sum_{n=1}^{N-1} |\text{sign}(s_n[n]) - \text{sign}(s_n[n-1])|$$

  The division by 2 ensures proper normalization. A high ZCR typically indicates unvoiced segments (noise or fricatives), while a low ZCR is typical of voiced segments (vowels). The threshold is determined adaptively using the mean zero-crossing rate across frames.

- **Energy:** Even if a frame has a low ZCR (potentially indicating voicing), it is classified as unvoiced if its energy falls below a minimum threshold. This is done because:
  - Low-energy segments often correspond to silence or background noise, which lack the harmonic structure of true voiced speech.
  - In speech processing, weak signals (even if periodic) contribute little to perceptual quality and are more efficiently modeled as noise-like unvoiced frames.
  - This prevents false voicing detection during pauses or weak fricatives (/h/ sounds).

2.2 Comparison of voiced and unvoiced frame plots

To visually distinguish the two types, different colors are used for the prediction error plots:

- Voiced frames are displayed in **dark green**
- Unvoiced frames are displayed in **dark violet**

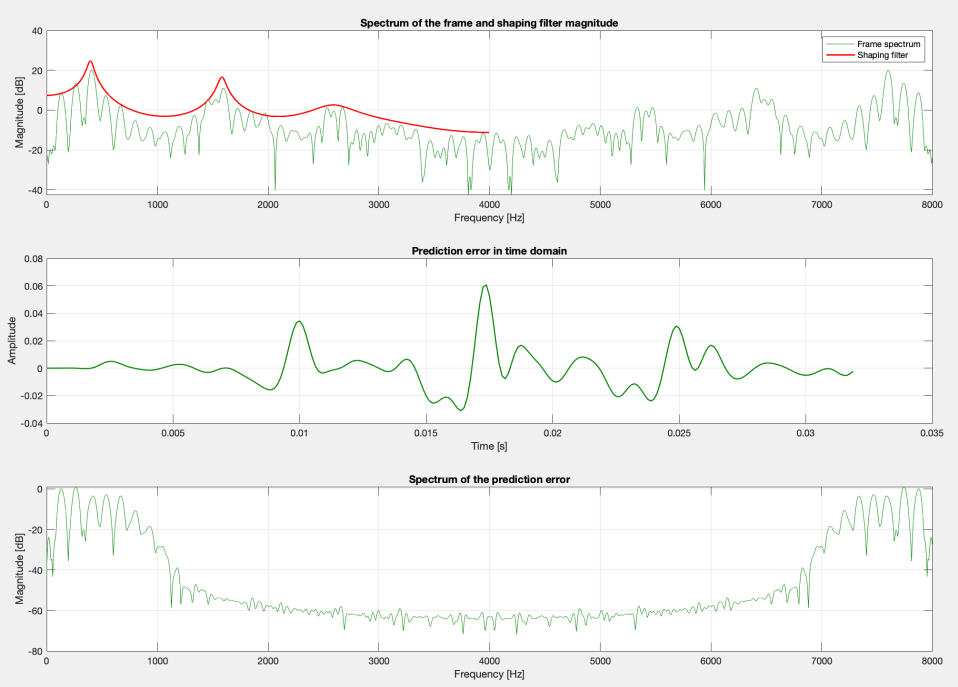

Figure 1: Plots of a voiced frame (dark green).

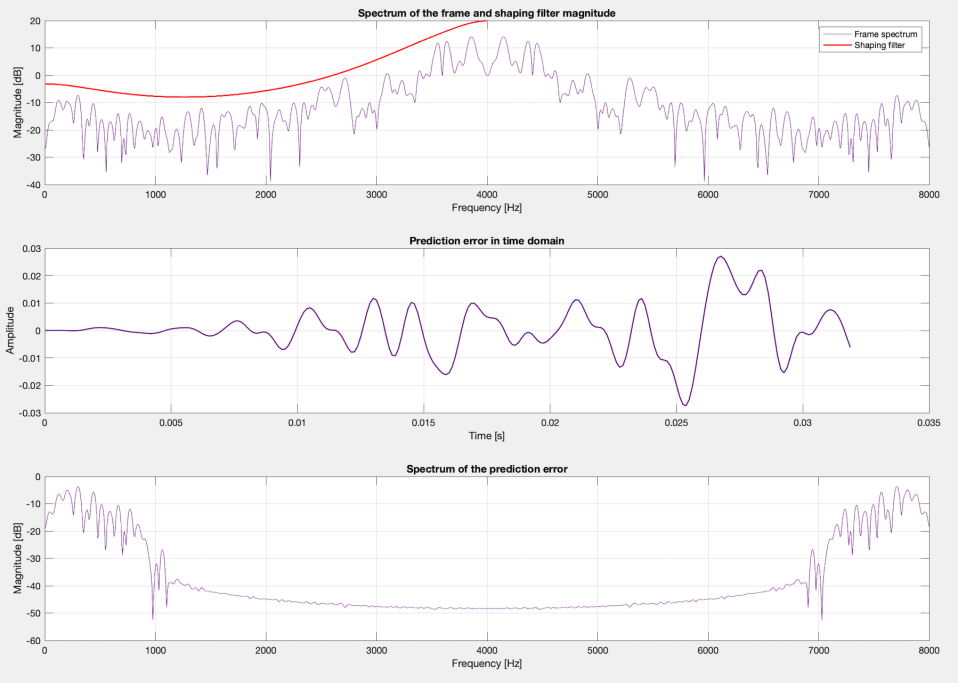

Figure 2: Plots of an unvoiced frame (dark violet).

The LPC analysis reveals clear differences between voiced and unvoiced frames. Figures 1 and 2 display the analysis for a typical voiced frame and a typical unvoiced frame, respectively.

For the **voiced frame** (Figure 1):

- **Spectrum of the frame and shaping filter:** The top plot exhibits distinct harmonic peaks and a clear spectral envelope, which is well modeled by the shaping filte. These harmonics reflect the periodic excitation due to vocal cord vibrations.
- **Prediction error in time domain:** The middle plot shows a quasi-periodic waveform, indicating a regular pitch period.
- **Prediction error spectrum:** The bottom plot presents a relatively flat spectrum with slight dips corresponding to formant frequencies, demonstrating that the LPC filter effectively whitens the signal.

For the **unvoiced frame** (Figure 2):

- **Spectrum of the frame and shaping filter:** The top plot lacks the clear harmonic structure seen in voiced frames. Instead, it presents a noisy, broadband spectrum. The shaping filter's envelope is smoother, reflecting the noise-like nature of unvoiced sounds.
- **Prediction error in time domain:** The middle plot exhibits an irregular, non-periodic signal, consistent with the random excitation of unvoiced sounds.
- **Prediction error spectrum:** The bottom plot shows a broad, unstructured spectrum indicative of a wideband noise.

2.3 Pitch detection with AMDF

Two common methods used for estimating the pitch period are the autocorrelation method and the Average Magnitude Difference Function (AMDF). Although both techniques analyze a windowed speech frame in the time domain, they differ in the metric used to evaluate periodicity.

The autocorrelation method calculates the correlation of the windowed signal with a time-shifted version of itself (multiplying them). By examining the resulting autocorrelation function, significant peaks indicate the lags where the signal best matches itself. The first major peak after the zero lag is associated with the pitch period. This approach is well known for its robustness, even in the presence of noise.

On the other hand, the AMDF method measures the average magnitude of the differences between the signal and its delayed version across various lags. Instead of searching for peaks, this method looks for minima in the

AMDF curve, where a lower value suggests a higher similarity between the compared segments. Although the AMDF is computationally simpler (it avoids multiplications), it can be more vulnerable to spurious local minima that do not correspond to the true pitch. In our case we use the AMDF:

The pitch detection algorithm computes the Average Magnitude Difference Function (AMDF) as

$$D_m = \frac{1}{L} \sum_{n=1}^{L} |x(n) - x(n-m)|,$$

where L is the frame length and m is the lag in samples. In our implementation, the function iterates over all lags m (from 1 to L), computing for each lag the average of the absolute differences between the signal sample $x(n)$ and its delayed version $x(n-m)$. For indices where $n-m$ is out of bounds, the signal is assumed to be zero (zero-padding). This yields a vector of AMDF values, where each element D_m quantifies the dissimilarity between the signal and its delayed version at lag m.

After computing D_m for all possible lags, we restrict our search for the minimum to a plausible range that typically captures the pitch period of human speech. In our implementation, the range of interest is between 25 and 80 samples. This choice is motivated by the following considerations:

- At a sampling rate of 8000 Hz, a lag of 25 samples corresponds approximately to a pitch frequency of

  $$f = \frac{8000}{25} \approx 320 \text{ Hz},$$

  which is close to the upper limit of typical human speech (especially for female voices).

- Conversely, a lag of 80 samples corresponds to

  $$f = \frac{8000}{80} = 100 \text{ Hz},$$

  a reasonable lower bound that is typical for male voices.

Thus, by selecting the lag that minimizes the AMDF in the range from 25 to 80 samples, we obtain a robust estimate of the pitch period.

The final pitch estimation is performed by:

```
pitch = find(amd == min(amd(25:80)));
```

which locates the index (the lag) where the AMDF reaches its minimum value within the specified range.

3. LPC Decoder

In the `decoder.m` script, the encoded parameters (LPC coefficients, gains, pitch values, etc.) are used to reconstruct the speech signal. The first step is to generate an excitation frame (see Section 3.1) using the encoded data, then filter it through the inverse of the whitening filter to synthesize speech. The decision to use an overlap-and-add method for reassembling the frames was made to ensure smooth transitions between consecutive frames, reducing artifacts in the reconstructed signal. Moreover, post-processing with a low-pass filter and a de-emphasis filter is applied to further refine the signal quality. Finally we evaluate the Signal Noise Ratio (SNR) and reproduce the reconstructed signal.

3.1 Generate excitation signal

The `generateexcitationsignal.m` function generates the excitation signal used during the decoding process. For voiced frames, a train of impulses with a period equal to the detected pitch is generated. For unvoiced frames, white noise is produced and then filtered to restrict its frequency content. The design choices here are based on:

- **Voiced frames:** Using a periodic impulse train replicates the periodic nature of voiced sounds (vowels).
- **Unvoiced frames:** The bandpass filtering of white noise (50 Hz to 3500 Hz) helps mimic the spectral characteristics of fricative sounds (*/s/, /z/, /v/, /f/, ecc...*).

Each excitation is then multiplied by its gain, that has been calculated prevoiusly in the encoding part.

4. Conclusions

4.1 Memory saved by LPC-10

In order to evaluate the memory savings offered by LPC-10 encoding (a key aspect in resource constrained devices like the speak and spell), we have added a section in the decoder that computes the total memory used for storing the LPC parameters. This is done by summing the sizes of each encoded component:

- **LPC Coefficients (A):** The total number of elements in all LPC coefficient vectors is multiplied by 8 bytes (assuming a double precision representation).
- **Gains and Pitches:** Similarly, the memory consumption for gains and pitch values is computed by considering each value as a double (8 bytes).
- **Voicing Flags:** These are stored as logical values, occupying 1 byte per flag.

The individual component sizes are summed to yield the total encoded size in bytes, which is then converted to kilobytes. This approach clearly demonstrates the efficiency of LPC coding by quantifying the amount of memory required to store the parameters compared to the original waveform. Below a table comparing the original dimension (kB) of three audio files and the compressed versions using LPC-10:

Table 1: LPC-10 compression

Filename	Dimension (kB)	
	Original	LPC-10
ces.wav	278	84
alphabet.mp3	1162	200
Invo.mp3	54	32

4.2 Low Pass Filtering of the prediction error

In order to improve the quality of the reconstructed speech, a low-pass filter is applied to the prediction error before further processing. The prediction error, which ideally should resemble white noise, may contain high frequency components and noise artifacts that can degrade both the pitch detection and the reconstructed signal quality. By low-pass filtering the prediction error (with a cutoff of 800 Hz in the encoder), we attenuate these unwanted high frequency components. This results in a cleaner residual signal, leading to

improved pitch estimation and, consequently, a higher quality reconstructed signal.

The effectiveness of this post-processing step is reflected in the Signal-to-Noise Ratio (SNR) of the reconstructed speech. Table 2 summarizes the SNR measurements obtained with and without the low-pass filter for three test files.

Table 2: Reconstruction SNR Comparison with and without Low-Pass Filtering of the Prediction Error

Filename	**Without LP Filter (dB)**	**With LP Filter (dB)**
ces.wav	-15.01	-8.48
invo.mp3	-14.62	-7.24
alphabet.mp3	-16.17	-8.89

As shown in table 2, the application of the low-pass filter leads to a significant improvement in the reconstruction SNR across all tested files. For example, in the case of `ces.wav`, the SNR improves from -15.03 dB to -8.47 dB when the LP filter is applied. Similar improvements are observed for the other two files. This demonstrates that filtering the prediction error effectively removes undesirable high-frequency noise and artifacts, resulting in a higher-quality LPC synthesis.